Lab 06 - Take a sad plot and make it better

Max Asmar

Introduction

Given below are two data visualizations that violate many data visualization best practices. Your task is to improve these visualizations using R and the tips for effective visualizations that we introduced in class. You should produce one improved visualization per dataset. Each visualization should be accompanied by a brief paragraph describing the choices you made in your improvement, specifically discussing what you didn't like in the original plots and why, and how you addressed them in the visualization you created.

This lab focuses on creative problem-solving! There are many ways to improve these visualizations - your job is to make thoughtful decisions and explain your reasoning.

Estimated time: 90-120 minutes

Learning Goals

By the end of this lab, you will be able to:

- Critique data visualizations using best practices
- Reshape data from wide to long format using `pivot_longer()`
- Improve plot aesthetics (titles, labels, colors, themes)
- Tell a clear story with data visualization
- Make design decisions and justify them
- Choose appropriate color palettes
- Modify plot elements to enhance readability

Prerequisites

Before you begin, make sure you have: - Completed Labs 01-05 - Reviewed lecture materials on data visualization best practices - Understand principles like: clear labels, appropriate colors, readable fonts, proper scales, effective use of space - Forked the lab-instructions repository

Review before starting: Make sure you're familiar with visualization principles discussed in lecture, including clarity, accuracy, and visual perception.

Getting Started

Navigate to your forked `lab-instructions` repository in JupyterHub, open the `Lab06` folder, and open the R Markdown document `lab-06.Rmd`.

Verify Your Setup

Before proceeding:

Step 1. Open `lab-06.Rmd` in RStudio

Step 2. Check that the Git pane shows YOUR username (not the course organization)

Step 3. Click **Knit** to make sure the document compiles

Warm Up

Before we introduce the data, let's warm up with some simple exercises.

Step 1: Update the YAML, changing the author name to your name.

Step 2: Knit the document.

Step 3: Commit your changes with message "Updated author name".

Step 4: Push to GitHub and verify the changes are visible in your repo.

Knit, commit with message "Completed warm up", and push your changes.

Packages

We'll use the **tidyverse** package for much of the data wrangling and visualisation and the data lives in the **dsbox** package. These packages are already installed for you. You can load them by running the following in your Console:

```
library(tidyverse)
library(dsbox)
```

Data

The datasets we'll use are called `instructors` and `fisheries` from the **dsbox** package. Since the datasets are distributed with the package, we don't need to load them separately; they become available to us when we load the package. You can find out more about the datasets by inspecting their documentation, which you can access by

running `?instructors` and `?fisheries` in the Console or using the Help menu in RStudio to search for `instructors` or `fisheries`. You can also find this information here and here.

Checkpoint: Run `glimpse(instructors)` and `glimpse(fisheries)` in your Console to verify the data loaded correctly.

Exercises

Part 1: Instructional Staff Employment Trends

The American Association of University Professors (AAUP) is a non-profit membership association of faculty and other academic professionals. This report compiled by the AAUP shows trends in instructional staff employees between 1975 and 2011, and contains an image very similar to the one given below.

What's wrong with this plot? Before we improve it, take a moment to think about what makes this visualization difficult to read or interpret.

The plot has an unlabeled x axis and y axis, plots too many variables on the same graph, and has no title. There should be a variable for year and a variable for faculty type.

Let's start by loading the data used to create this plot.

```
staff <- read_csv("data/instructional-staff.csv")
```

Each row in this dataset represents a faculty type, and the columns are the years for which we have data. The values are percentage of hires of that type of faculty for each year.

```
## # A tibble: 5 x 12
##   faculty_type    `1975` `1989` `1993` `1995` `1999` `2001` `2003` `2005` `2007`
##   <chr>            <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>
## 1 Full-Time Tenu~   29     27.6   25     24.8   21.8   20.3   19.3   17.8   17.2
## 2 Full-Time Tenu~   16.1   11.4   10.2    9.6    8.9    9.2    8.8    8.2    8
## 3 Full-Time Non-~   10.3   14.1   13.6   13.6   15.2   15.5   15     14.8   14.9
## 4 Part-Time Facu~   24     30.4   33.1   33.2   35.5   36     37     39.3   40.5
## 5 Graduate Stude~   20.5   16.5   18.1   18.8   18.7   19     20     19.9   19.5
## # i 2 more variables: `2009` <dbl>, `2011` <dbl>
```

Reshaping the Data

In order to recreate this visualization we need to first reshape the data to have one variable for faculty type and one variable for year. In other words, we will convert the data from **wide format** to **long format**.

Graduate Student Employees
Part-Time Faculty
Full-Time Non-Tenure-Track Faculty
Full-Time Tenure-Track Faculty
Full-Time Tenured Faculty
5
10
15
20

But before we do so, a thought exercise: *How many rows will the long-format data have?*

Think about it: The data will have a row for each combination of year and faculty type. If there are 5 faculty types and 11 years of data, how many rows will we have?

Your answer: The long-format data will have 55 rows.

Calculation: 5 faculty types × 11 years = 55 rows

We do the wide to long conversion using a new function: `pivot_longer()`. The animation below shows how this function works, as well as its counterpart `pivot_wider()`.

```
## NULL
```

The function has the following arguments:

```
pivot_longer(data, cols, names_to = "name", values_to = "value")
```

- The first argument is `data` as usual
- The second argument, `cols`, is where you specify which columns to pivot into longer format
- The third argument, `names_to`, is the name of the new column that will contain the old column names
- The fourth argument, `values_to`, is the name of the new column that will contain the values

Here's how we use it for our data:

```
staff_long <- staff %>%
  pivot_longer(
    cols = -faculty_type,              # All columns except faculty_type
    names_to = "year"                   # Column names become "year"
  ) %>%
  mutate(year = as.numeric(year))      # Convert year to numeric
```

What this code does:

- Takes the `staff` dataset
- Pivots all columns except `faculty_type` into long format
- Creates a new column called `year` containing the old column names (1975, 1989, etc.)
- Creates a new column called `value` (default name) containing the percentages
- Converts `year` from character to numeric so we can plot it as a continuous variable

- Saves the result as `staff_long`

Let's take a look at what the new longer data frame looks like.

```
staff_long
```

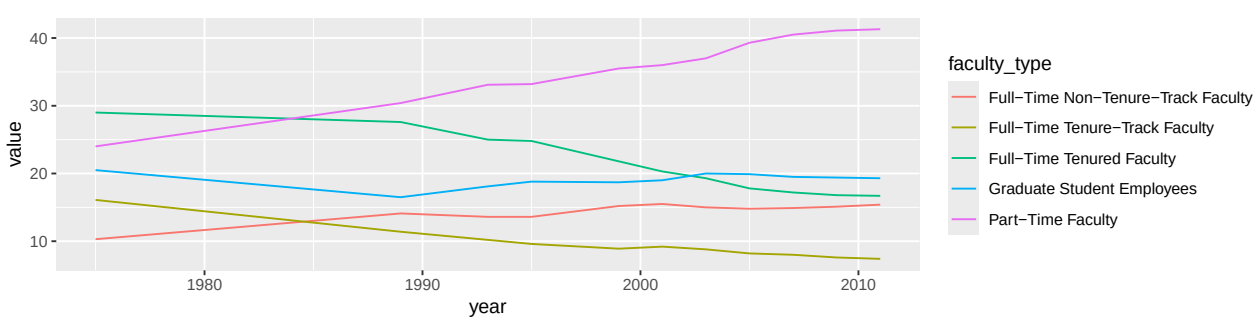
```
## # A tibble: 55 x 3
##    faculty_type                year value
##    <chr>                      <dbl> <dbl>
##  1 Full-Time Tenured Faculty  1975  29
##  2 Full-Time Tenured Faculty  1989  27.6
##  3 Full-Time Tenured Faculty  1993  25
##  4 Full-Time Tenured Faculty  1995  24.8
##  5 Full-Time Tenured Faculty  1999  21.8
##  6 Full-Time Tenured Faculty  2001  20.3
##  7 Full-Time Tenured Faculty  2003  19.3
##  8 Full-Time Tenured Faculty  2005  17.8
##  9 Full-Time Tenured Faculty  2007  17.2
## 10 Full-Time Tenured Faculty  2009  16.8
## # i 45 more rows
```

Notice: Now instead of 5 rows and 12 columns, we have 55 rows and 3 columns!

Creating the Line Plot

Now let's plot it as a line plot. A possible approach for creating a line plot where we color the lines by faculty type is the following:

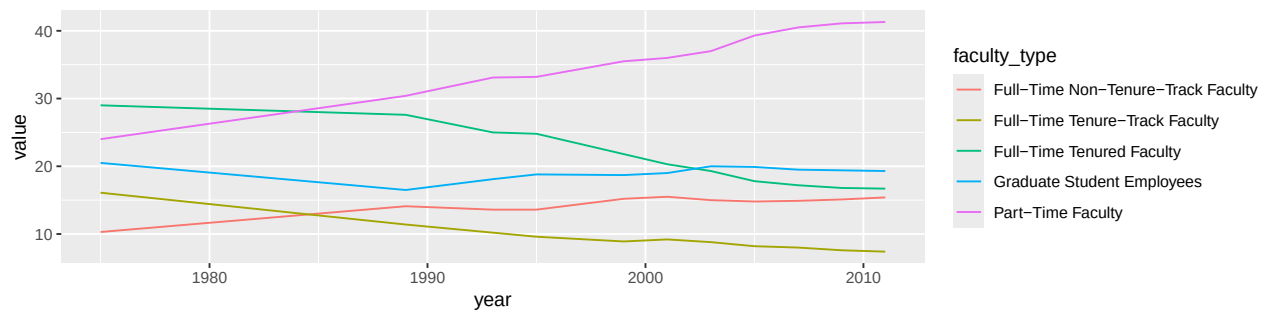
```
staff_long %>%
  ggplot(aes(x = year, y = value, color = faculty_type)) +
  geom_line()
```



What happened? You'll notice this results in a warning message about groups. This is because ggplot doesn't know that each faculty type should be its own line.

We can fix this using the `group` aesthetic:

```
staff_long %>%
  ggplot(aes(x = year,
             y = value,
             group = faculty_type,      # Tell ggplot to group by faculty type
             color = faculty_type)) +   # Color by faculty type
  geom_line()
```



What this code does:

- `x = year`: Time goes on the x-axis
- `y = value`: Percentage goes on the y-axis
- `group = faculty_type`: Each faculty type gets its own line
- `color = faculty_type`: Each faculty type gets its own color
- `geom_line()`: Draw lines connecting the points

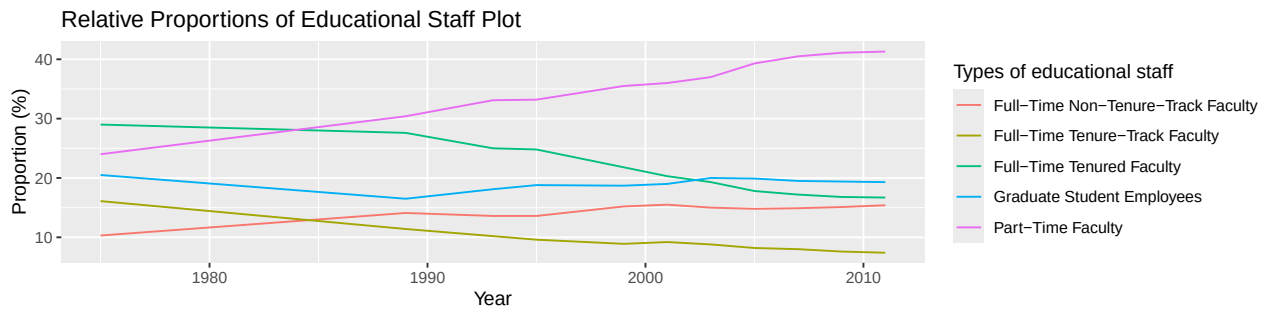
Exercise 1

Include the line plot you made above in your report and make sure the figure width is large enough to make it legible. Also add a title, improve the axis labels, and fix the legend label.

Add these improvements to your plot:

```
# Remember to change eval=FALSE to eval=TRUE!
staff_long %>%
  ggplot(aes(x = year,
             y = value,
             group = faculty_type,
             color = faculty_type)) +
  geom_line() +
  labs(
    title = "Relative Proportions of Educational Staff Plot",
    x = "Year",
```

```
    y = "Proportion (%)",
    color = "Types of educational staff"
  )
```



Hints for good labels:

- Title should describe what the plot shows
- X-axis: What does year represent?
- Y-axis: What are these values? (Include units!)
- Legend: "faculty_type" is code - what's a better label for users?

Knit, commit with message "Completed Exercise 1", and push your changes.

Exercise 2

Suppose the objective of this plot was to show that the proportion of part-time faculty have gone up over time compared to other instructional staff types. What changes would you propose making to this plot to tell this story? Why would these changes help?

Brainstorm improvements: Consider the following aspects:

Visual emphasis:

- How could you highlight the part-time faculty line?
- Should other lines be de-emphasized?

Color choices:

- Would different colors help tell the story?
- Should part-time faculty be a bold/bright color?

Additional elements:

- Would annotations help?
- Should you add reference lines or labels?

Layout/scales:

- Is the y-axis scale appropriate?
- Would the plot be clearer with a different aspect ratio?

Your proposed changes (write at least 3):

- Change 1: Emphasize part-time faculty by changing their color to red and changing all the other trendlines to be gray.

  Why: The change in percentage of Part-Time faculty is the most important finding from this graph, so making the other trendlines gray would help move the readers' eyes towards the Part-Time faculty trendline.

- Change 2: Adding annotations to the bottom of the graph would help readers identify patterns more easily.

  Why: Since there are 5 trendlines on the graph it would be helpful to summarize the graph for the reader and identify major findings, such as the big increase in percentage of part-time faculty.

- Change 3: Increasing the size of the lines, especially the part-time faculty line, would help emphasize the differences in percentage of educational staff

  Why: The trendlines would be easier to trace and help distinguish each staff type from each other. The change in proportion of part-time faculty is the most prominent observation and would be easier to read if the line was bolded.

Exercise 3

Implement the changes you proposed in the previous exercise.

Here are some possible approaches to consider:

To highlight one line:

- Use a bright color for part-time faculty and gray for others
- Make the part-time line thicker: `geom_line(aes(size = faculty_type))`
- Use `gghighlight` package (if you want to explore)

To change colors manually:

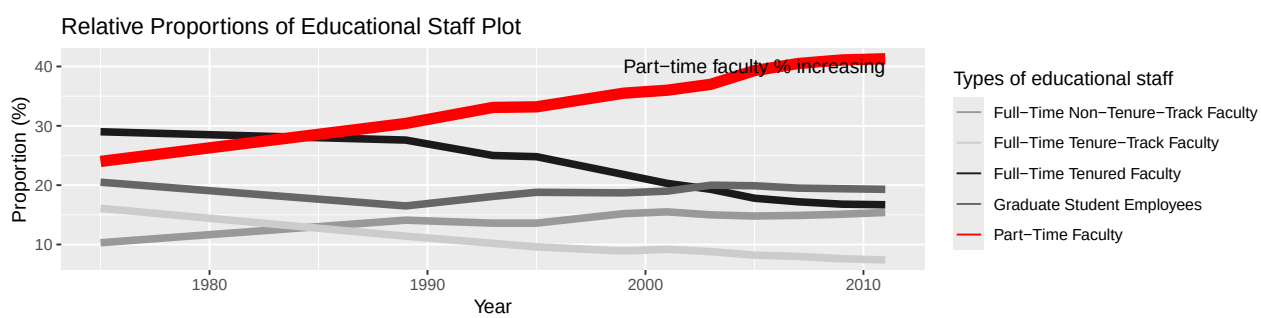
```
scale_color_manual(values = c("Part-Time Faculty" = "red",
                              "Full-Time Tenured Faculty" = "gray60",
                              # ... etc
                              ))
```

To add annotations:

```
annotate("text", x = 2005, y = 40, label = "Part-time faculty rising")
```

Your improved visualization:

```
staff_long %>%
  ggplot(aes(x = year,
             y = value,
             group = faculty_type,
             color = faculty_type)) +
  geom_line(aes(linewidth = faculty_type))  +
  guides(linewidth= "none") +
  scale_linewidth_manual(values=c(
    "Part-Time Faculty" = 3,
    "Full-Time Tenured Faculty" = 2,
    "Graduate Student Employees" = 2,
    "Full-Time Non-Tenure-Track Faculty" = 2,
    "Full-Time Tenure-Track Faculty" = 2
  )) +
  annotate("text", x = 2005, y = 40, label = "Part-time faculty % increasing") +
  scale_color_manual(values = c("Part-Time Faculty" = "red",
                                "Full-Time Tenured Faculty" = "gray10",
                                "Graduate Student Employees" = "gray40",
                                "Full-Time Non-Tenure-Track Faculty" = "gray60",
                                "Full-Time Tenure-Track Faculty" = "gray80"
                                )) +
  labs(
    title = "Relative Proportions of Educational Staff Plot",
    x = "Year",
    y = "Proportion (%)",
    color = "Types of educational staff"
  )
```



Explain your design decisions: Write a brief paragraph (3-5 sentences) describing the changes you made and how they help tell the story that part-time faculty proportions have increased.

I colored the part-time faculty trendline to be red while making all of the other lines shades of gray to emphasize the change in proportion of part-time faculty members. In the top right corner of the graph, I added an annotation to demonstrate that there was a positive net change in the percentage of part-time faculty out of educational staff. I also increased the line width so that the trendlines for all independent variables would be easier to follow. I think visibility is one of the most important aspects of good data visualizations, and the contrast between the bright red of the part-time faculty trendline and the rest of the trendlines helps tell the story of how part-time faculty proportions increased over time.

Knit, commit with message "Completed Exercises 2-3", and push your changes.

Part 2: Fisheries

Fisheries and Aquaculture Department of the Food and Agriculture Organization of the United Nations collects data on fisheries production of countries. This Wikipedia page lists fishery production of countries for 2016. For each country tonnage from capture and aquaculture are listed. Note that countries whose total harvest was less than 100,000 tons are not included in the visualization.

A researcher shared with you the following visualization they created based on these data:

What's wrong with this plot? Look at it carefully and think about readability, colors, labels, and effectiveness.

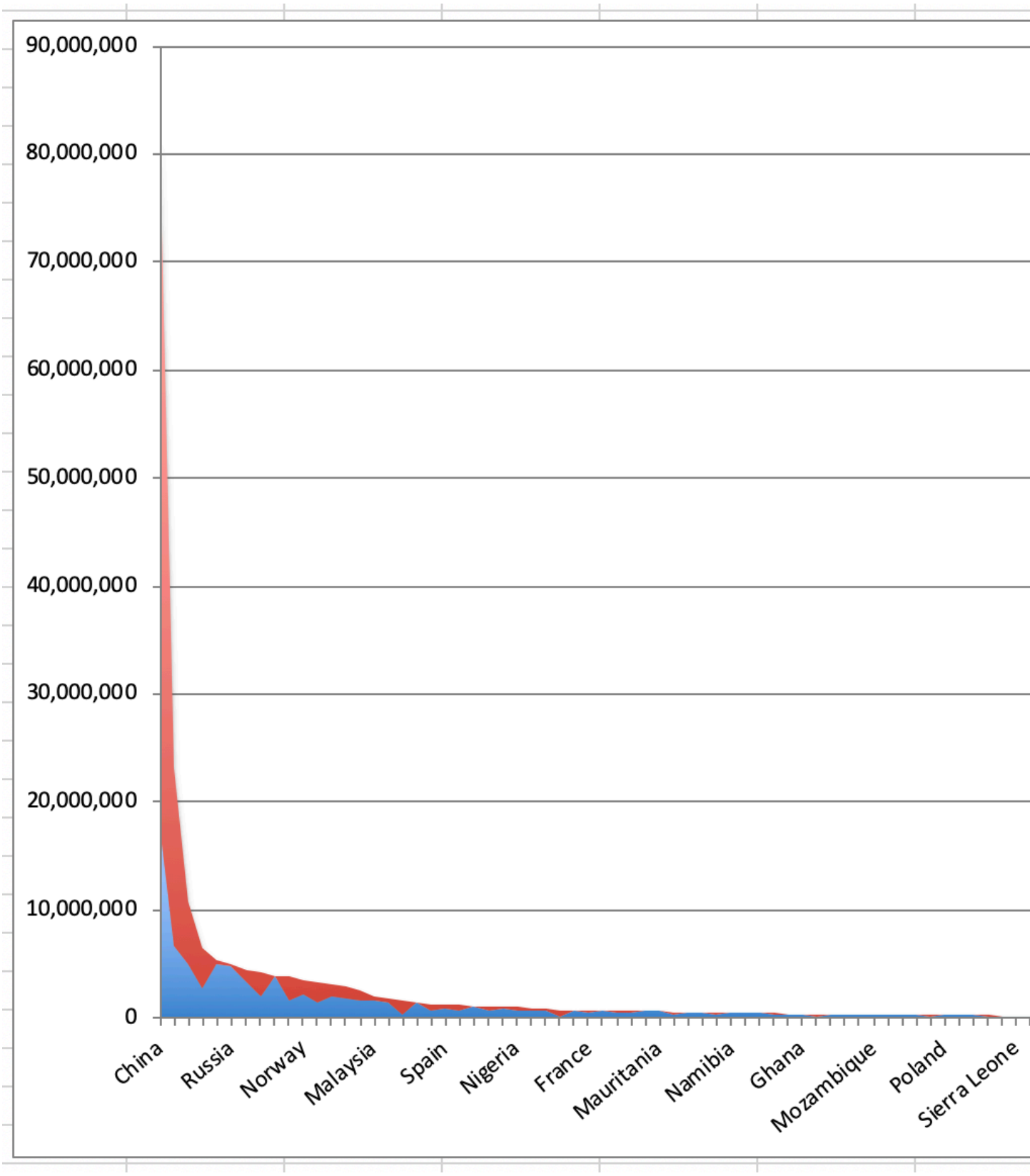
The readability of the plot could be greatly improved because many of the variables appear to overlap and there are many unnecessary features. For example, it is distracting that the pie charts are not flat. There are no axis labels or title for the chart on the left.

Exercise 4

Can you help them improve it? First, brainstorm how you would improve it. Write up the changes/decisions you would make as bullet points. It's ok if some of your improvements are aspirational (you don't know how to implement them yet, but you think they're good ideas).

Critique the current visualization: What specific problems do you see?

Problems with the original plot:


90,000,000
80,000,000
70,000,000
60,000,000
50,000,000
40,000,000
30,000,000
20,000,000
10,000,000
0
China
Russia
Norway
Malaysia
Spain
Nigeria
France
Mauritania
Namibia
Ghana
Mozambique
Poland
Sierra Leone

- Problem 1: The pie charts are distracting because they are not flat (2D in spatial coordinates).
- Problem 2: There are no axis labels on the area chart.
- Problem 3: There is no title for the area chart.
- Problem 4: The only country that is interpretable in the area chart is China while the scale is inappropriate for the rest of the countries in the study.

Proposed improvements: For each problem above, suggest a solution:

- Improvement 1: The pie charts should be turned into flat (2D in spatial coordinates) charts so that the charts are more intuitive and easy to read.
- Improvement 2: The area chart should have axis labels for both the x and y axis.
- Improvement 3: The area chart should have a title, which can be added inside the labs() function.
- Improvement 4: The area chart should be filtered to just include the top 20 countries in the dataset.

Possible issues to consider:

- Are the country names readable?
- Is the color scheme effective?
- Are the axis labels clear?
- Is the data ordered in a useful way?
- Does the plot type effectively show the data?
- Are there too many/too few elements?

First, let's load the data:

```
fisheries <- read_csv("data/fisheries.csv")
```

```
## Rows: 216 Columns: 4
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (1): country
## dbl (3): capture, aquaculture, total
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

Explore the data: Run these commands in your Console to understand the data:

```
glimpse(fisheries)
View(fisheries)
```

Data structure: Notice that this data has columns for country, capture, aquaculture, and total. Think about how you might want to reshape or filter this data for your visualization.

Exercise 5

Create a new data visualisation for these data that implements the improvements you proposed in the previous exercise (or as many of them as you can).

Some suggestions to consider (you don't have to use all of these!):

Reordering data:

```
fisheries %>%
  mutate(country = fct_reorder(country, total))  # Order by total
```

Filtering for top countries:

```
fisheries %>%
  slice_max(total, n = 20)  # Keep only top 20
```

Better color palettes:

```
scale_fill_viridis_d()          # Colorblind-friendly
scale_fill_brewer(palette = "Set2")  # ColorBrewer palettes
```

Horizontal bars for readability:

```
coord_flip()  # Makes bars horizontal so country names are readable
```

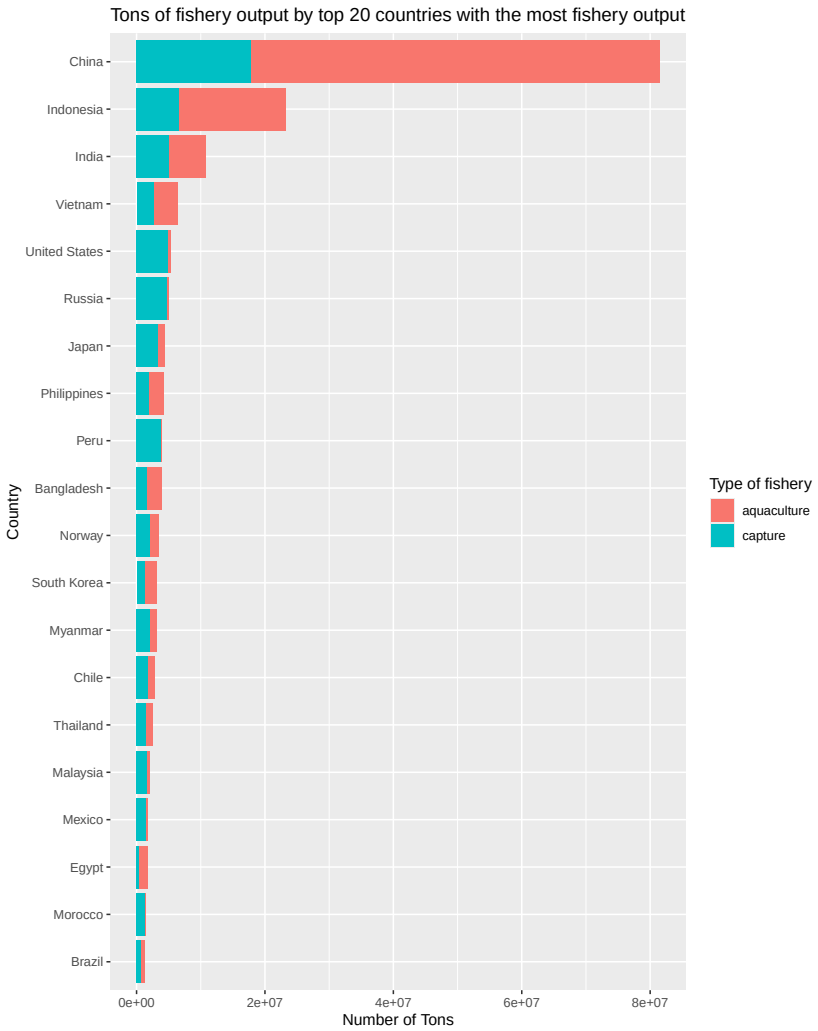
Clearer labels:

```
labs(title = "...", x = "...", y = "...", fill = "...")
```

Your improved visualization:

```
# Remember to change eval=FALSE to eval=TRUE!
# Your code here
  fisheries %>%
    slice_max(total, n = 20) %>% # Keep only top 20
    mutate(country = fct_reorder(country, total)) %>%
```

```
  pivot_longer(
    cols=c(capture, aquaculture),
    names_to = "fishery_type",
    values_to = "ton_amount"
  ) %>%
ggplot(aes(x = country,
           y = ton_amount,
           fill = fishery_type)) +
geom_col() +
labs(
  title = "Tons of fishery output by top 20 countries with the most fishery output",
  x = "Country",
  y = "Number of Tons",
  fill = "Type of fishery"
) +
  coord_flip()
```



Explain your design decisions: Write a paragraph (4-6 sentences) describing the changes you made and how they improve the visualization.

I decided to create a variable to represent the type of fishery so that I could have a tidier dataset and so that I could use this new variable (fishery_type) to use as fill color for the graph. For the graph, I decided to use a bar chart to show both the relative proportions of aquaculture v. capture in each country and the approximate number of tons for each type of fishery. I also added axis labels for both the number of tons (x axis) and the country (y axis). I used coord_flip() so that the country names would fit on the visualization. My graph is also 2 spatial dimensions, which is different than the pie charts from before. I think my graph combines the findings from all three of the original graphs because of the fill and bar_chart type. I think having multiple levels of analysis on the same plots overall helps for a cleaner aesthetic and stronger visualization.

Did your improved visualization accomplish your goals? Are there any improvements you proposed that you weren't able to implement? If so, what are they?

One improvement I proposed that I wasn't able to implement was improving the pie chart by making it two dimensional. I felt like the bar chart was the best way to summarize the findings from all 3 charts, but I'm still curious to explore pie charts and their benefits and disadvantages in presenting data.

Knit, commit with message "Completed Exercises 4-5", and push your changes.

Common Errors and Troubleshooting

Data reshaping errors:

- **Error with `pivot_longer()`** → Make sure `cols` specifies the columns to pivot (use `-column_name` to exclude a column)
- **Data looks weird after pivot** → Check that `names_to` and `values_to` are appropriate for your data
- **Year showing as character** → Use `mutate(year = as.numeric(year))` to convert

Plotting errors:

- **Lines not connecting** → Make sure you have `group =` aesthetic for line plots
- **Legend labels are ugly** → Use `labs(color = "...")` or `labs(fill = "...")` to rename
- **Colors are hard to distinguish** → Try `scale_color_viridis_d()` or ColorBrewer palettes
- **Country names overlap** → Use `coord_flip()` to make horizontal, or rotate with `theme(axis.text.x = element_text(angle = 45))`

Manual color errors:

- **Error: "Insufficient values"** → Make sure you provide a color for each level of your variable
- **Colors not showing** → Check that your color names match the levels in your data exactly (case-sensitive!)

Figure sizing:

- **Plot too small** → Adjust `fig.width` and `fig.height` in chunk options

- **Plot too large** → Reduce `fig.width` and `fig.height`
- **Aspect ratio weird** → Try `fig.asp` instead of `fig.height`

General issues:

- **Code chunk not running** → Make sure you changed `eval=FALSE` to `eval=TRUE`
- **Can't find data** → Check that the `data` folder is in the correct location
- **Improvements not showing** → Make sure you're actually adding the new code to your plot, not just the original plot

---

Key insight: Good data visualization is about more than just making plots - it's about making *effective* plots that communicate clearly and accurately. Every design choice (colors, labels, ordering, scales) should serve the goal of helping your audience understand the data.

Design principle: When improving visualizations, always ask: "What story am I trying to tell?" and "What makes this easier or harder for my audience to understand?"

More Ugly Charts

Want to see more ugly charts and practice your critique skills?

- Flowing Data - Ugly Charts
- Reddit - Data is ugly
- Missed Opportunities and Graphical Failures
- (Mostly Bad) Graphics and Tables

To submit to Canvas:

Step 1. In RStudio, click the **Knit** dropdown menu (next to the Knit button)

Step 2. Select **Knit to tufte_handout** to generate a PDF

Step 3. Download the PDF file from the Files pane

Step 4. Upload the PDF to Canvas

Final Checkpoint: Visit your GitHub repo one more time to confirm all your work is there. We will grade what we see in your repo on GitHub!